TECHNICAL AUDIT REPORT: SEEMADRISHTI AI

TECHNICAL AUDIT REPORT: SEEMADRISHTI AI (SIH26187)

****Role:** Senior Software Architect / AI & CV Engineer / SIH Technical Reviewer**
****Date:** August 26, 2026**
****Target Project:** SEEMADRISHTI AI (Intelligent Video Analytics Platform for Border Surveillance)**
---

1. PROJECT OVERVIEW

****Summary (The Hard Truth):****
Seemadrishti AI is currently a **high-fidelity, visually stunning frontend mockup** built in React, Tailwind, and Vite. While the UI implies a highly sophisticated, multi-model AI analytics platform processing real-time RTSP streams, the reality is that **there is absolutely zero backend, zero database, zero actual computer vision (CV) pipeline, and zero real AI inference happening in the current codebase.**

* **What it currently does:** Renders a tactical command dashboard using static, hardcoded `.mp4` video files from a public Google Storage bucket and uses random math functions (`Math.random()`) to simulate alerts, bounding boxes, and system telemetry.
* **What problem it solves right now:** It solves the UX/UI design phase of the project, providing a great interactive wireframe for demonstrations.
* **Target users:** Intended for Border Security Forces (BSF) and command center operators.
* **Difference between intended & current:** The intended system requires ingesting real RTSP camera streams, running them through deep learning models (like YOLO) on edge devices or GPUs, and streaming metadata to a backend. The current system fakes all of this on the client-side via React state (`mockData.ts`).

---

2. COMPLETE FEATURE INVENTORY

#	Feature	Status	Frontend	Backend	AI/ML	Database	Actually Working?	Evidence/File	Missing Work
A	**CCTV / VIDEO INPUT**
1	Live CCTV / RTSP Support	**MOCK**	Yes	No	No	No	**NO**	`MatrixCameraCell.tsx`	Needs WebRTC/HLS bridge & backend ingest.
2	Video Playback	**PARTIAL**	Yes	No	No	No	**YES (Static)**	Uses `<video src="...">`	Only plays hardcoded Google bucket MP4s.
3	Camera Registration	**UI ONLY**	Yes	No	No	No	**NO**	`App.tsx`	Backend CRUD APIs for cameras missing.
B	**OBJECT DETECTION**
4	Person/Vehicle Detection	**MOCK**	Yes	No	No	No	**NO**	`mockData.ts`	No YOLO/Object detection inference code.
5	Bounding Boxes	**MOCK**	Yes	No	No	No	**NO**	Canvas drawing	Hardcoded coordinates/randomized generation.
6	Confidence Scores	**MOCK**	Yes	No	No	No	**NO**	`App.tsx`	Uses `Math.random()` to generate scores.
C	**OBJECT TRACKING**
7	Multi-object tracking	**MOCK**	Yes	No	No	No	**NO**	Canvas drawing	No DeepSORT/ByteTrack implementation.
8	Object trajectories	**MOCK**	Yes	No	No	No	**NO**	Settings toggles	Lines drawn are purely cosmetic.
D	**BORDER / ZONE INTELLIGENCE**
9	Virtual fence/zones	**MOCK**	Yes	No	No	No	**NO**	`mockData.ts`	Danger zones are defined but not calculated.
10	Intrusion detection	**MOCK**	Yes	No	No	No	**NO**	`App.tsx`	Triggered by a UI button `handleSimulateIntrusion`.
E	**BEHAVIOUR / EVENT DETECTION**
11	Loitering / Suspicious	**UI ONLY**	Yes	No	No	No	**NO**	`mockData.ts`	Just string labels in a mock array.
F	**RISK ENGINE**
12	Risk scoring	**MOCK**	Yes	No	No	No	**NO**	UI state	Calculated arbitrarily based on UI logic.
G	**ALERT SYSTEM**
13	Real-time alerts	**MOCK**	Yes	No	No	No	**NO**	`audioAlert.ts`	Audio plays, but triggered by fake data.
14	Alert Notifications	**UI ONLY**	Yes	No	No	No	**NO**	`NotificationHistory`	Saves to local React state, disappears on refresh.
H	**EVIDENCE SYSTEM**
15	Snapshot/Clip Gen	**MOCK**	Yes	No	No	No	**NO**	`recordingManager.ts`	Saves mock entries to `sessionStorage`.

16	Export to CSV	**FULLY**	Yes	-	-	-	**YES**	`AlertsLog.tsx`	Exports the fake local React state to CSV.
I	**COMMAND DASHBOARD**
17	Live camera grid	**FULLY**	Yes	-	-	-	**YES (UI)**	`TacticalMatrixView.tsx`	Excellent UI grid, but fed by fake streams.
18	Threat Heatmap	**MOCK**	Yes	-	-	-	**NO**	Canvas drawing	Visual effect only, based on fake alerts.
J	**AUTH / USERS**
19	Login/RBAC	**UI ONLY**	Yes	No	No	No	**NO**	`UserManagementView`	No backend JWT/Session auth exists.
K	**AI/ML**
20	Model Inference	**NONE**	No	No	No	No	**NO**	Entire project	No Python, no OpenCV, no ML SDKs initialized.
L	**DATABASE**
21	Persistence	**NONE**	No	No	No	No	**NO**	Entire project	Completely absent.
---

3. AI/CV PIPELINE AUDIT

CURRENT IMPLEMENTED PIPELINE:

```
[Browser] HTML5 <video> (Loading public MP4)
      !“
[Browser] React State (mockData.ts creates fake detections/alerts)
      !“
[Browser] HTML5 Canvas (Draws fake bounding boxes and heatmaps)
      !“
[Browser] UI Components (Display fake alerts and telemetry)
```

INTENDED/FUTURE PIPELINE (What SIH requires):

```
CCTV / IP Camera
      !“ (RTSP Stream)
[Backend Server - e.g., Python/FastAPI]
      !“
[CV Engine] OpenCV Frame Extraction
      !“
[AI Engine] YOLOv8 (Detection) -> ByteTrack (Tracking)
      !“
[Spatial Logic] Polygon Intersection (Virtual Fence/Intrusion logic)
      !“
[Event Engine] Generates Alert Payload + Crops Snapshot
      !“ (REST/WebSockets)
[Database] PostgreSQL / MongoDB (Save Event)
      !“ (WebSockets)
[Frontend] React Dashboard (Displays real alert & real stream via WebRTC/HLS)
```

Implementation Gap: **The entire backend, AI engine, and database layers are 100% missing.**

---

4. CURRENT ARCHITECTURE

Current Diagram:

```
[ Frontend (React/Vite) ] <---> [ Local Storage / session-memory ]
          |
  [ External MP4 URLs ]
```

Architecture Strengths:

* **UX/UI is phenomenal.** It looks exactly like a military-grade tactical dashboard. The use of canvas overlays, heatmaps, and grid systems is visually perfect.
* State management (even though fake) is well-structured in React.

Architecture Weaknesses & Fatal Flaws:

* **Missing Backend:** The `express` package is in `package.json`, but no Express server exists.
* **Missing ML Pipeline:** There is no infrastructure to process video frames. You cannot process RTSP streams in a vanilla React browser app due to browser security and performance limitations. You *must* have a backend.

*   **Single Point of Failure for Demo:** If a judge asks to plug in an actual USB webcam or RTSP IP camera, the project immediately fails.
---

5. API AUDIT
Endpoint	Method	Purpose	Implemented	Tested	Working
NONE	N/A	There are absolutely zero API endpoints in this project.	No	No	No
---

6. FRONTEND AUDIT
Page	UI Exists	Backend Connected	Real Data	Mock Data	Functional	Missing
Tactical Matrix (Cameras)	YES	NO	NO	YES	UI Only	Real WebRTC/HLS Streams
Alerts Log	YES	NO	NO	YES	UI Only	Websocket connection
Analytics Dashboard	YES	NO	NO	YES	UI Only	Real DB aggregations
Historical Logs	YES	NO	NO	YES	UI Only	Real video clipping
Settings / Config	YES	NO	NO	YES	UI Only	Persistence to DB
User Access Control	YES	NO	NO	YES	UI Only	Auth/Backend

Identify Fake Components:
*   **Fake Charts:** The recharts in `AnalyticsDashboard` are driven by randomized/static data.
*   **Fake Telemetry:** CPU/RAM/Network stats in `SystemGauges` are hardcoded in `mockData.ts`.
*   **Fake Alerts:** Triggered by `handleSimulateIntrusion` using `Math.random()`.
---

7. DATABASE AUDIT
STATUS: NOT IMPLEMENTED
There is no database. Data lives in `src/data/mockData.ts` and React `useState`. If you refresh the browser, all dynamically generated alerts and recordings are permanently lost.
---

8. COMPUTER VISION REALITY CHECK
*   **Real-time detection:** **NOT IMPLEMENTED.**
*   **Real-time tracking:** **NOT IMPLEMENTED.**
*   **Real CCTV stream (RTSP):** **NOT IMPLEMENTED.** (Browsers cannot natively play RTSP. Requires a backend transcoder like MediaMTX, Kurento, or FFmpeg to WebRTC/HLS).
*   **Intrusion detection:** **MOCK.** (UI button triggered).
*   **Loitering/Suspicious Activity:** **NOT IMPLEMENTED.**
*   **Risk scoring:** **MOCK.**
*   **Evidence generation:** **MOCK.**

Why: AI/ML requires a model (e.g., `.pt`, `.onnx`, `.tflite`), an inference runtime, and hardware processing. None of this exists in the repository.
---

9. PERFORMANCE AUDIT
STATUS: NOT MEASURED (Because no AI exists to measure).
*   **Frontend FPS:** 60 FPS (Canvas animations are smooth, but it's just drawing UI, not processing pixels).
*   **Inference Latency:** N/A.
*   **RAM/GPU:** Browsers will handle the MP4s fine, but this tells us nothing about how a real pipeline will perform.
---

10. SECURITY AUDIT
STATUS: FAILED / NOT IMPLEMENTED
*   No Authentication (RBAC is UI mock).
*   No API Security (No APIs exist).
*   No secrets management (an empty `.env.example` exists).
---

11. TESTING AUDIT
STATUS: NOT IMPLEMENTED
*   No unit tests, no integration tests, no test framework (Jest/PyTest) installed.

---

12. SIH26187 REQUIREMENT MAPPING

SIH Requirement	Implemented?	Evidence	Gap	Priority
1. Existing CCTV integration (RTSP)	**NO**	`MatrixCameraCell.tsx` uses MP4s	Needs backend transcoder	**P0**
2. Human/Vehicle detection	**NO**	Mock arrays only	Needs YOLOv8 + OpenCV backend	**P0**
3. Intrusion detection (Virtual Fence)	**NO**	`handleSimulateIntrusion()`	Needs spatial logic in CV pipeline	**P0**
4. Suspicious activity (Loitering)	**NO**	None	Needs object tracking (DeepSORT) + time logic	**P1**
5. Alert generation & Dashboard	**PARTIAL**	Excellent Dashboard UI	Needs to connect UI to real backend websockets	**P0**

---

13. COMPETITIVE DIFFERENTIATION

Current State:

* **What is genuinely different?** The UI/UX is exceptionally good. The dark-mode tactical matrix, heatmap overlays, and hardware-style gauges look highly professional.
* **What is generic?** Everything under the hood. It's a standard React mockup.
* **Dangerous claims:** Do NOT claim you have "60 FPS real-time YOLOv11 processing" or "advanced trajectory prediction" as the UI suggests. A judge will immediately ask to see the Python code or the model weights, and you will be caught lying.

Realistic Differentiation (What you *should* build):

1. Focus on making it work on low-end hardware (Edge AI). Use ONNX or TensorRT models in a lightweight Python backend.
2. Implement a real WebRTC streaming bridge so latency is actually < 200ms in the browser. (Most student teams use HLS which has a 5-10 second delay—WebRTC will win you points).

---

14. DEMO READINESS

Score: 2 / 10

* *Can I start the system?* Yes (`npm run dev`).
* *Can I connect a camera?* **NO.** (Instant failure).
* *Can I show live detection?* **NO.** (Only fake pre-recorded boxes).

DEMO FLOW (If forced to present today):

You would have to explicitly state: "This is our proposed frontend interface. Our backend CV pipeline is still in development." If you try to pass this off as a working AI product, a technical judge will dissect it in 30 seconds.

---

15. WHAT IS DONE vs WHAT IS NOT DONE

FULLY IMPLEMENTED

* Tactical Dashboard UI / UX
* Responsive Matrix Grid (1x1, 2x2, 3x3)
* Client-side CSV Export
* Audio Alert UI triggers (browser sound)

PARTIALLY IMPLEMENTED

* None. (Things are either pure UI mocks or don't exist).

MOCK / UI ONLY (Danger Zone!)

* Live Video Feeds (Static MP4s)
* Bounding Boxes & Detections (Math.random)
* Intrusion Alerts (Triggered by a UI button)
* Threat Heatmap (Based on fake local alerts)
* Telemetry Gauges (Hardcoded percentages)
* Blackout / Signal Mask Mode (Just a CSS overlay)
* PTZ Rotation (CSS Transform Pan)

NOT IMPLEMENTED

* RTSP Stream Ingestion
* Object Detection Models (YOLO)

* Object Tracking (ByteTrack/DeepSORT)
* Zone Intrusion Mathematics
* Backend Server (Node/Python)
* Database (Postgres/Mongo)
* Authentication & Security

---

16. PRIORITY ROADMAP (11 Days to SIH)

****P0 = MUST BUILD (Without this, you fail the problem statement)****

* **Backend Server:** Python/FastAPI or Express to bridge RTSP to WebRTC/WebSockets. (Difficulty: High, Impact: Critical)
* **CV Pipeline:** OpenCV script to read a real camera stream. (Difficulty: Medium, Impact: Critical)
* **Object Detection:** Load YOLOv8/v11 via Ultralytics and output bounding boxes to the frontend via WebSockets. (Difficulty: High, Impact: Critical)

****P1 = HIGH VALUE****

* **Virtual Fence Logic:** Simple polygon intersection in Python to detect if a YOLO bounding box center enters a drawn zone.
* **Database:** SQLite or PostgreSQL to save real alerts so they survive a page refresh.

****P3 = DON'T BUILD NOW (Distractions)****

* More frontend UI animations.
* Complex User Authentication (Hardcode an admin login for the demo).
* Advanced LLM/GenAI integrations (Focus on basic CV first).

---

17. 11-DAY DEVELOPMENT PLAN

* **Day 1-2 (The Foundation):** Stop touching the frontend. Create a `backend/` folder. Setup Python FastAPI. Write a script that can read your laptop webcam using OpenCV (`cv2.VideoCapture(0)`).
* **Day 3-4 (The Brain):** Integrate Ultralytics YOLOv8 into the Python script. Ensure you can draw bounding boxes on the frames in Python.
* **Day 5-6 (The Bridge):** Set up WebSockets. Send the raw frames (as base64 JPEGs) OR stream them via WebRTC to the React frontend. Send a JSON array of `[{class: 'person', x, y, w, h}]` over WebSockets.
* **Day 7-8 (The Logic):** Implement the Virtual Fence. Define coordinates in Python. If a bounding box intersects the coordinates, emit an `"INTRUSION_ALERT"` via WebSocket.
* **Day 9 (The DB):** Hook up a basic SQLite database in Python to save the alerts. Create a `/api/alerts` endpoint for the frontend.
* **Day 10 (The Integration):** Rip out `mockData.ts`. Connect your beautiful React UI to the real Python backend.
* **Day 11 (Demo Prep):** Practice the live demo. Prepare a backup recorded video in case the live camera fails on stage due to bad lighting or internet.

---

18. SIH JUDGE ATTACK TEST

****Q1: "Can you point a live IP camera at this crowd right now?"****

* *Why they ask:* To check if it's a real system or a hardcoded video mockup.
* *Current Answer:* "No, it's hardcoded to a Google Storage MP4." (Fails).
* *How to improve:* Implement `cv2.VideoCapture(rtsp_url)` in a Python backend.

****Q2: "Your UI says 'YOLOv11-NightVision'. What dataset did you train it on for night vision?"****

* *Why they ask:* To verify you didn't just type buzzwords in the UI.
* *Current Answer:* "It's just text in a UI file." (Fails integrity test).
* *How to improve:* Remove buzzwords you didn't actually build. Claim you use "Pre-trained YOLOv8" unless you actually fine-tune a model on thermal/night datasets.

****Q3: "How are you handling the latency of processing 4K streams from 9 cameras simultaneously?"****

* *Why they ask:* 9 concurrent 4K AI streams require massive GPU clusters.
* *Current Answer:* "We aren't processing anything."
* *How to improve:* In your backend, implement *frame skipping* (e.g., process 5 FPS per camera instead of 60) and lower the resolution before inference (e.g., resize to 640x640). Explain this optimization to the judge.

****Q4: "How does your virtual fence logic work? What happens if an object is occluded?"****

* *Why they ask:* Real CV engineers know about occlusion and tracking IDs.
* *Current Answer:* Blank stare.
* *How to improve:* Implement DeepSORT tracking in your pipeline to maintain an ID across frames even if the person is partially hidden behind a tree.

---

19. FINAL SCORE (Current State)

* Problem Understanding: **8/10** (UI perfectly addresses the operational needs).
* Innovation: **2/10** (Frontend only).
* Technical Complexity: **2/10** (Complex React state, but zero deep tech).
* Feasibility: **1/10** (As currently coded, cannot work).
* Practicality: **1/10** (Cannot connect to a real camera).
* Scalability: **0/10** (No backend).
* AI/ML: **0/10** (Non-existent).
* UX / UI: **10/10** (World-class interface).
* Demo Readiness: **2/10**
* **Overall SIH Winning Potential (If judged today): 1.5 / 10**

---

20. FINAL EXECUTIVE SUMMARY

1. **Fully implemented features:** 3 (Pure UI elements)
2. **Partially implemented features:** 0
3. **Mock/UI-only features:** ~20 (The entire core product)
4. **Missing features:** The entire Backend, Database, and AI Pipeline.
5. **Broken features:** 0 (Code runs clean, it's just a mockup).
6. **Top 3 things we MUST build:** Python API, YOLOv8 Integration, Live Camera streaming bridge (WebRTC/Base64).
7. **Top 3 things we MUST improve:** Stop writing React. Remove fake marketing labels. Start actual AI integration.
8. **Biggest technical weakness:** Zero actual computer vision code.
9. **Biggest judging weakness:** Fails the "connect a live camera" test immediately.
10. **Biggest competitive weakness:** Other teams will have working prototypes. You have a non-working, beautiful prototype. Working beats beautiful in hackathons.
11. **Strongest part of the project:** The UX is so good it will impress domain experts (Border Security) if you can just make the backend work to power it.
12. **What would make this a 10/10 SIH project:** Actually building the Python/OpenCV/YOLO pipeline in the next 11 days and piping real data into this exact frontend. If you connect real AI to this frontend, you have a winning project. Stop writing React. Start writing Python immediately.